数据表示、位级运算与浮点数 2

2.1 进位计数制与位串解释

位串的真值由位宽、编码和小数点位置共同确定。1 B=8 bit，B 表示字节，b 表示位。

$$(a_k \cdots a_0.a_{-1} \cdots)_r = \sum_i a_i r^i, \qquad 0 \le a_i < r.$$

转换	规则
十进制整数转 r 进制	除 r 取余，余数倒序排列
十进制小数转 r 进制	乘 r 取整，整数顺序排列
二进制与八进制	从小数点向两侧每 3 位一组；整数端左补 0，小数端右补 0
二进制与十六进制	从小数点向两侧每 4 位一组，补 0 规则同上

有限十进制小数可有无限二进制展开，如 0.1；有限位存储时须按规定截断或舍入。

例 2-1

若十进制数为 137.5，则其八进制数为（　）。

A. 89.8　　B. 211.4　　C. 211.5　　D. 1011111.101

2.2 定点数编码与范围

真值是数学数值，机器数是约定编码下的位串。定点表示约定固定小数点位置。

n 位整数编码	数值规则	范围与零
无符号	全部位为正权	$0 \sim 2^n - 1$
原码	最高位符号，其余为绝对值	$-(2^{n-1}-1) \sim 2^{n-1}-1$，两个零
反码	正数同原码，负数数值位逐位取反	同原码范围，两个零
补码	最高位权 -2^{n-1}，其余正权	$-2^{n-1} \sim 2^{n-1}-1$，一个零

补码值：

$$x = -b_{n-1}2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i = \begin{cases} u, & u < 2^{n-1}; \\ u - 2^n, & u \ge 2^{n-1}, \end{cases}$$

其中 u 为该位串的无符号值。负数补码可由同宽绝对值逐位取反加 1 得到。8 位 `FDH` 表示 -3，`80H` 表示 -128。最小负数的数学相反数超出同宽补码范围，同宽取反加 1 会保留原位串。

原码的 -0 为 `1000⋯0`，反码的 -0 为 `1111⋯1`。一位符号、$n-1$ 位小数时，原码、反码范围为 $[-(1-2^{-(n-1)}), 1-2^{-(n-1)}]$；补码范围为 $[-1, 1-2^{-(n-1)}]$。

整数移码采用偏置 2^{n-1} 时，$E = x + 2^{n-1}$，其编码由同宽补码翻转最高位得到。IEEE 阶码采用另一偏置，按字段定义计算。

2.3 截断、扩展与 C 类型转换

变换	位级规则与条件
缩短位宽	题设截断模型保留低位；保值需被删除高位满足原值扩展关系
无符号扩展	高位补 0
补码扩展	高位复制原符号位
转 n 位无符号数	按模 2^n 的值转换
补码扩大后保值	新增高位均为原符号位

源数的解释决定扩展方式。16 位无符号 65530 扩为 32 位是 `0000 FFFAH`；16 位补码 -32767 转 32 位无符号是 $2^{32}-32767$，位串 `FFFF 8001H`。

例 2-2

假定编译器规定 int 型和 short 型长度分别为 32 位和 16 位，执行下列 C 语言语句：

```c
unsigned short x = 65530;
unsigned int y = x;
```

得到 y 的机器数为（　）。

A. 0000 7FFAH　　B. 0000 FFFAH　　C. FFFF 7FFAH　　D. FFFF FFFAH

例 2-3

在 32 位计算机上执行下列 C 语言代码段后，ui 的值是（　）。

```c
short si = -32767;
unsigned int ui = si;
```

A. $2^{15}-1$　　B. $2^{15}+1$　　C. $2^{32}-2^{15}-1$　　D. $2^{32}-2^{15}+1$

C 表达式先作整数提升和通常算术转换，再确定比较与运算类型。32 位 `int i=-1; unsigned u=1;` 的常见模型中，`i<u` 为假，因 i 转换为 $2^{32}-1$。

本章 C11/C17 口径下：无符号运算按模 2^n；有符号算术溢出属未定义行为；超范围转有符号类型涉及实现规定。题设机器的截断、绕回模型应与语言保证分别使用。

逻辑移位与算术移位

操作	补入位与保值条件
逻辑右移	高位补 0，无符号值除 2^k 向下取整
补码算术右移	高位补符号位，值除 2^k 向负无穷取整
左移	低位补 0；高位丢失可能导致溢出

8 位补码 -3 算术右移 1 位为 -2；C 整数 $-3/2$ 向零截断为 -1。C11/C17 负有符号数右移涉及实现规定，使用题设算术右移模型时须明确该条件。左移按位结果与 C 有符号左移的合法性分别检查。

2.4 逻辑门、选择器与三态门

A, B	与 AB	或 $A+B$	异或 $A \oplus B$
0,0	0	0	0
0,1	0	1	1
1,0	0	1	1
1,1	1	1	0

逻辑非、与、或的通常优先级依次降低；括号及非号覆盖范围优先。德摩根律：$\overline{A+B}=\bar{A}\bar{B}$，$\overline{AB}=\bar{A}+\bar{B}$。多个位异或为 1，当且仅当其中 1 的个数为奇数；可用于本位和与奇偶校验。

k 选 1 MUX 至少需 $\lceil \log_2 k \rceil$ 位选择编码；2 选 1：$Y=\bar{s}D_0+sD_1$。32 位 MUX 同时选择 32 条对应位。

三态门关闭时输出高阻态，开启时驱动 0 或 1。高阻表示不主动驱动公共线。共享总线同一时刻须满足驱动源控制约束，多个源输出相反电平会争用。

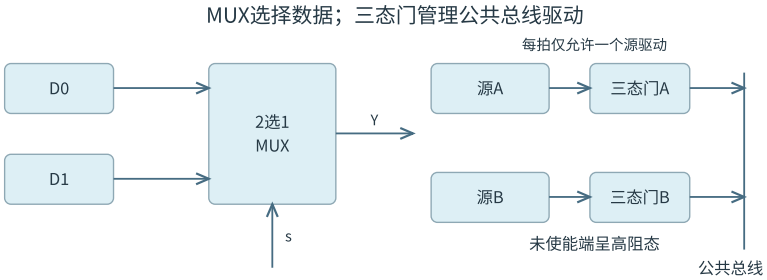


Figure 2.1: 选择器选择数据源，三态门控制公共线驱动

2.5 加减运算器与进位逻辑

规定 C_i 为第 i 位输入进位，C_{i+1} 为输出进位：

$$S_i = A_i \oplus B_i \oplus C_i, \qquad G_i = A_i B_i, \quad P_i = A_i \oplus B_i,$$

$$C_{i+1} = G_i + P_i C_i.$$

例如 $C_2 = G_1 + P_1G_0 + P_1P_0C_0$。串行进位逐级传播；先行进位用组合逻辑展开进位，增加电路复杂度以缩短关键路径。“并行输入各位” 与 “并行生成进位” 是两项实现性质。

加减控制位 Sub 令第二输入为 $B \oplus \text{Sub}$、最低进位 $C_0 = \text{Sub}$：

$$\text{Sub} = 0:\ A + B; \qquad \text{Sub} = 1:\ A + \overline{B} + 1 = A - B \pmod{2^n}.$$

ALU 按操作控制选择算术、与、或、异或等功能，移位器位置以具体通路为准。数据位宽与操作选择编码位宽分别确定。

例 2-4

某计算机字长为 8 位，CPU 中有一个 8 位加法器。已知无符号数 $x = 69, y = 38$，若在该加法器中计算 $x - y$，则加法器的两个输入端信息和输入的低位进位信息分别为（　）。

A. 0100 0101、0010 0110、0
B. 0100 0101、1101 1001、1
C. 0100 0101、1101 1010、0
D. 0100 0101、1101 1010、1

2.6 运算标志与条件转移

本章采用 CF=1 表示减法借位的约定。设 n 位加减器最高输出进位为 C_n：

$$\text{CF} = C_n \oplus \text{Sub}, \quad \text{OF} = C_{n-1} \oplus C_n, \quad \text{SF} = S_{n-1}, \quad \text{ZF} = [S = 0].$$

CF 判断无符号溢出或借位；OF 判断补码有符号溢出。同号相加结果异号时加法溢出；异号相减结果与被减数异号时减法溢出。采用 “无借位” 进位约定的 ISA 应按其定义改判 CF。

8 位运算	低 8 位	CF	OF	SF	ZF
`7FH+01H`	`80H`	0	1	1	0
`FFH+01H`	`00H`	1	0	0	1
`80H-01H`	`7FH`	0	1	0	0
`00H-01H`	`FFH`	1	0	1	0

计算 $a - b$ 后的比较条件：

条件	标志表达式
相等	ZF=1
无符号 $a < b$	CF=1
无符号 $a > b$	CF=0 且 ZF=0
有符号 $a < b$	SF ≠ OF
有符号 $a > b$	SF=OF 且 ZF=0

例 2-5

假定有符号整数采用补码表示，若 int 型变量 x 和 y 的机器数分别是 FFFF FFDFH 和 0000 0041H，则 x、y 的值及 x−y 的机器数分别是（　）。

A. x=−65，y=41，x−y 的机器数溢出
B. x=−33，y=65，x−y 的机器数为 FFFF FF9DH
C. x=−33，y=65，x−y 的机器数为 FFFF FF9EH
D. x=−65，y=41，x−y 的机器数为 FFFF FF96H

例 2-6

已知 x、y 为 int 类型，当 x=100、y=200 时，执行 “x 减 y” 指令得到的溢出标志 OF 和借位标志 CF 分别为 0、1。那么当 x=10、y=−20 时，执行该指令得到的 OF 和 CF 分别是（ ）。

A. OF=0，CF=0　B. OF=0，CF=1　C. OF=1，CF=0　D. OF=1，CF=1

2.7 无符号乘法与乘法器组织

两个 n 位数完整乘积最多需 $2n$ 位。无符号顺序乘法初态 $P=0$，Y 装乘数，X 装被乘数。每轮：检查 $Y_0 \to$ 若为 1 则 $P \leftarrow P+X$ 并保留进位 $\to (C,P,Y)$ 整体逻辑右移；共 n 轮。

$X=1101, Y=1011$ 时：

轮次	Y_0；加法后 C,P	右移后 P,Y
初态	—	0000,1011
1	1；0,1101	0110,1101
2	1；1,0011	1001,1110
3	0；0,1001	0100,1111
4	1；1,0001	1000,1111

结果 `10001111`=143。若只保存低 n 位，高 n 位非零表示无符号乘积溢出。

乘法器组织	特征
顺序乘法器	复用加法器与移位器，跨多周期完成
阵列乘法器	并行生成并压缩部分积，延迟由组合路径决定
流水线乘法器	多级寄存器分隔组合路径，多组运算可重叠，提高吞吐率

常数乘法可在语义与位宽允许时化为移位加减，如 $13x=(x\ll 3)+(x\ll 2)+x$。

Booth 补码一位乘法

乘数后附 $q_{-1}=0$，每轮观察 (Y_0,q_{-1})：00、11 加 0，01 加 X，10 减 X；随后 (P,Y,q_{-1}) 整体算术右移。符号位参加运算，跨寄存器边界传位。

$X=1101=-3, Y=0110=6$：

轮次	观察位与操作	右移后 P,Y,q_{-1}
初态	—	0000,0110,0
1	00，加 0	0000,0011,0
2	10，减 X	0001,1001,1
3	11，加 0	0000,1100,1
4	01，加 X	1110,1110,0

完整乘积 `11101110`=−18。保留低 n 位补码时，未溢出条件为完整积的高 n 位均为低半部符号位的扩展，即完整积最高 $n+1$ 位全部相同。最小负数的取负及中间操作须配置足够保护位。

2.8 除法、商溢出与余数校正

无符号双精度被除数以 (R,Q) 保存，各 n 位，除数 Y 为 n 位。单精度被除数先零扩展。除数 $Y>0$ 时，初始高半部 $R\geq Y$ 意味着商至少为 2^n，产生商溢出。4 位商中，255/15=17 溢出，200/15=13 余 5 可表示。

算法	每轮更新与结束条件
恢复余数法	移入下一被除数位，试减除数；非负保留差、商位 1；负则加回除数、商位 0
不恢复余数法	根据上一轮余数符号决定本轮减或加除数；按新余数生成商位；最终负余数须按规则校正

无符号最终结果必须满足 $N=QY+R$ 且 $0\leq R<Y$。试减的暂时负值需符号或借位状态支持。

恢复余数法的寄存器轨迹

15/2 初态 $R=0000, Q=1111, Y=0010$，每轮先整体左移 RQ，再试减并记录商位，共 4 轮，得到 $Q=0111, R=0001$。

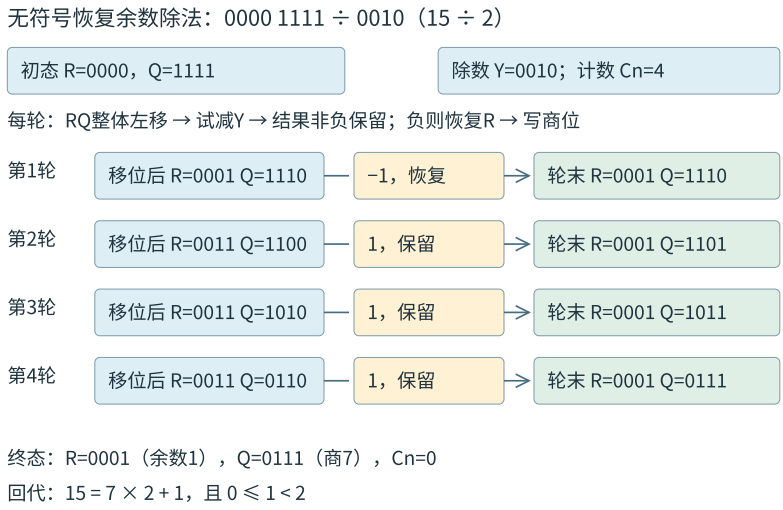


Figure 2.2: 15 除以 2 的试减、恢复和商位写入

不恢复余数法的 7/3，取 4 位有符号 R、3 位 Q：

轮次	整体左移后 R	本轮操作	操作后 R	记商后 Q
初态	—	—	0	111
1	1	减 3	−2	110
2	−3	加 3	0	101
3	1	减 3	−2	010

末轮 $R<0$，再加 3 校正为 1；商 2、余 1。

有符号除法与异常

按向零截断约定，商符号由两操作数符号异或确定，非零余数与被除数同号，$X = QY + R$。例如 $-15/2$ 的商 -7、余 -1；$15/(-2)$ 的商 -7、余 1。

课件 4 位补码 $-7/3$：被除数符号扩展为 $R = 1111, Q = 1001, Y = 0011$。本例负余数试加正除数；试加改变原负号时恢复，否则保留并生成商的绝对值位。

轮次	左移后 R、Q	试加 R	轮末 R、Q
1	1111、0010	0010，恢复	1111、0010
2	1110、0100	0001，恢复	1110、0100
3	1100、1000	1111，保留	1111、1001
4	1111、0010	0010，恢复	1111、0010

最后商绝对值 2，异号取补为 1110=-2；余数 1111=-1。整除时需单独正确接受试算零。也可在足够宽的内部表示中先求绝对值，完成无符号除法后恢复符号。

除数 0 须异常处理；$-2^{n-1}/(-1)$ 超出 n 位补码商范围。固定轮数除法器按位宽迭代；提前退出须由题设明确给出。

2.9 IEEE 754 字段与特殊值

设尾数字段整数值为 F、位数为 p、阶码为 E、偏置为 B，符号为 s。

参数	binary32	binary64
符号／阶码／尾数字段位数	1 ／ 8 ／ 23	1 ／ 11 ／ 52
偏置 B	127	1023
规格化有效精度	24 位	53 位
规格化真阶码	–126～127	–1022～1023
最小正规格化数	2^{-126}	2^{-1022}
最小正次正规数	2^{-149}	2^{-1074}
最大有限正数	$(2-2^{-23})2^{127}$	$(2-2^{-52})2^{1023}$

字段条件	数值／含义
$E=0, F=0$	正零或负零，由 s 区分
$E=0, F\neq 0$	次正规数（非规格化数），$(-1)^s(F/2^p)2^{1-B}$，隐含前导 0
E 既非全 0 也非全 1	规格化数，$(-1)^s(1+F/2^p)2^{E-B}$，隐含前导 1
E 全 1，$F=0$	正无穷或负无穷
E 全 1，$F\neq 0$	NaN

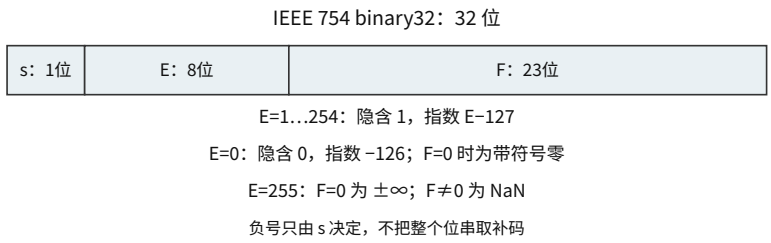


Figure 2.3: 单精度字段与指数编码

binary32 位串 `00000001H` 为 2^{-149}，`00800000H` 为 2^{-126}。编码前先规格化绝对值，最后独立写符号位；负浮点数按符号、指数、尾数解读。比较有限同号负数时，绝对值较大者数值较小。

例 2-7

-0.4375 的 IEEE 754 单精度浮点数表示为（ ）。

A. BEE0 0000H　B. BF60 0000H　C. BF70 0000H　D. C0E0 0000H

例 2-8

float 型数据常用 IEEE 754 单精度浮点格式表示。假设两个 float 型变量 x 和 y 分别存放在 32 位寄存器 f1 和 f2 中，若 `(f1)=CC90 0000H`、`(f2)=B0C0 0000H`，则 x 和 y 之间的关系为（ ）。

A. x<y 且符号相同　B. x<y 且符号不同　C. x>y 且符号相同　D. x>y 且符号不同

2.10 浮点加减与舍入

顺序为：对阶 → 尾数加减 → 规格化 → 舍入 → 检查规格化与阶码范围。

步骤	规则与边界
对阶	小阶向大阶对齐，小阶尾数右移；最终阶码为原合法大阶，可能损失低位
右规	尾数出现 $10.xxx_2$ 时右移 1 位、阶码加 1，可能上溢
左规	抵消后左移尾数、阶码减少，可能触及次正规区或下溢边界；精确零单独处理
舍入	可能使尾数进位，再右规并检查上溢
渐进下溢	指数过小时可能形成次正规数，接近 0 后才舍入为零

最近偶数舍入：被舍部分小于半个间隔取低端，大于半个间隔取高端，恰好居中取保留有效数最低位为 0 的候选。保护位 G、舍入位 R、粘滞位 S 保留舍入依据，S 为更低被舍位的逻辑或；最近偶数下，$G=1$ 且 $(R \vee S \vee \text{LSB})=1$ 时向保留部分进 1。

例 2-9

下列有关浮点数加减运算的叙述中，正确的是（ ）。

I. 对阶操作不会引起阶码上溢或下溢
II. 右规和尾数舍入都可能引起阶码上溢
III. 左规时可能引起阶码下溢
IV. 尾数溢出时，结果不一定溢出

A. 仅 II、III　B. 仅 I、II、IV　C. 仅 I、III、IV　D. I、II、III、IV

浮点加法可能不满足结合律。binary32 取 $a=2^{25}, b=-2^{25}, c=1$，则 $(a+b)+c=1$，而 $b+c$ 先舍入回 b，得到 $a+(b+c)=0$。

浮点精度与数值范围

binary32 在 $[2^e, 2^{e+1})$ 中的相邻正规格化数间隔为 2^{e-23}。所有 $[-2^{24}, 2^{24}]$ 内整数可精确表示，界外部分整数仍可精确表示；在 $[2^{24}, 2^{25})$ 中间隔 2，在 $[2^{25}, 2^{26})$ 中间隔 4。

精确数学结果	binary32 最近偶数结果
$2^{24}+0.5$	16777216
$2^{24}+1$	16777216
$2^{24}+1.5$	16777218
$(2^{24}+2)+1$	16777220

精确性由有效位与舍入决定，有限范围由指数和尾数上界共同决定。32 位补码在其范围内表示连续整数；binary32 的数值范围较大、有效精度为 24 位。

例 2-10

已知 $f(n)=\sum_{i=0}^{n}2^i=2^{n+1}-1=\underbrace{11\cdots1}_{n+1\text{位}}{}_2$，计算 $f(n)$ 的 C 语言函数 f1 如下。

```c
int f1(unsigned n) {
    int sum = 1, power = 1;
    for (unsigned i = 0; i <= n - 1; i++) {
        power *= 2;
        sum += power;
    }
    return sum;
}
```

将 f1 中的 int 都改为 float，可得到计算 $f(n)$ 的另一个函数 f2。假设 unsigned 和 int 型数据都占 32 位，float 采用 IEEE 754 单精度标准。请回答下列问题。

(1) 当 n=0 时，f1 会出现死循环，为什么？若将 f1 中的变量 i 和 n 都定义为 int 型，则 f1 是否还会出现死循环？为什么？

(2) f1(23) 和 f2(23) 的返回值是否相等？机器数各是什么（用十六进制表示）？

(3) f1(24) 和 f2(24) 的返回值分别为 33 554 431 和 33 554 432.0，为什么不相等？

(4) $f(31)=2^{32}-1$，而 f1(31) 的返回值却为 -1，为什么？若使 f1(n) 的返回值与 f(n) 相等，则最大的 n 是多少？

(5) f2(127) 的机器数为 7F80 0000H，对应的值是什么？若使 f2(n) 的结果不溢出，则最大的 n 是多少？若使 f2(n) 的结果精确（无舍入），则最大的 n 是多少？

32 位 int 转 binary64 可精确，转 binary32 可能丢低位。binary32 有限值扩为 binary64 可精确保留，已经丢失的信息无法恢复。浮点转整数向零截断且须检查目标范围；NaN 与任何值含自身的相等比较为假。

2.11 字节序、边界对齐与数组地址

存放规则	最低地址保存的字节
大端	最高有效字节；12345678H 依次 12、34、56、78
小端	最低有效字节；12345678H 依次 78、56、34、12

字节序只改变一个多字节对象内部各字节的位置，字节内位序、数组元素先后次序保持各自规定。−6 的 16 位补码 FFFAH 按小端存为 FA、FF。

自然对齐模型下，2 B 对象起点为 2 的倍数，4 B 对象起点为 4 的倍数。成员前填充满足各成员对齐，结构末尾填充满足相邻数组元素对齐，具体规则由题设或 ABI 给出。

成员偏移可按 $\lceil d/a \rceil a$ 对齐，其中 d 为前一成员结束后的偏移，a 为当前成员对齐要求。结构总长按结构对齐要求取整。

例 2-11

某计算机存储器按字节编址，采用小端方式存放数据。假定编译器规定 int 型和 short 型长度分别为 32 位和 16 位，并且数据按边界对齐存储。某 C 语言程序段如下：

```
struct {
    int a;
    char b;
    short c;
} record;
record.a = 273;
```

若 record 变量的首地址为 0xC008，则地址 0xC008 中的内容及 record.c 的地址分别为（　）。

A. 0x00，0xC00D　　B. 0x00，0xC00E

C. 0x11，0xC00D　　D. 0x11，0xC00E

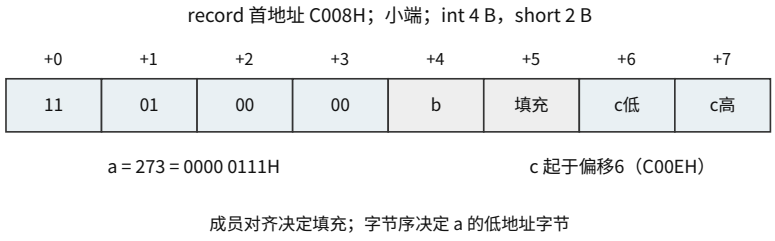


Figure 2.4: 结构体成员与填充位置

按字节编址、元素宽 w B、行优先二维数组列数 C：

$$\text{addr}(a[i][j]) = \text{base}(a) + (iC + j)w.$$

`struct { int id; char name[10]; int salary; }` 在 int 为 4 B 且自然对齐时，id 偏移 0，name 偏移 4，salary 偏移 16，结构长 20 B。数组首址 A0B0H，第二元素 id=12345678H 时，小端字节 56H 位于 A0C5H。

机器字长、存储字长与指令字长分别确定。按字节编址的 32 位整数占 4 个地址，按 32 位字编址则占 1 个地址；容量与跨界访问均先统一地址单位。